

Manifold-Aligned Guided Integrated Gradients for Reliable Feature Attribution

Le-Van Thai

September 2026

1 Overall

- Feature attribution methods: gán một importance score cho mỗi input feature.
- path-based methods: hấp dẫn vì liên kết attributions với các yêu cầu có cơ sở lý thuyết như completeness và sensitivity.
- IG can yield unreliable explanations when the integration path between a baseline and the input passes through regions with noisy gradients.
- GIG:
 - GIG giải quyết được vấn đề của IG về noisy gradient, nhưng: $GIG \rightarrow \text{input space}$ $\rightarrow$ có thể lệch data manifold
 - GIG dùng straight-line anchors để kéo path trở lại, nhưng: straight-line anchors $\rightarrow$ lại phụ thuộc noisy straight-line trajectory
 - EIG/MIG chuyển sang latent space để cải thiện manifold alignment, nhưng lại có vấn đề khác: latent-space path $\rightarrow$ plausible manifold nhưng $\rightarrow$ không quan tâm logit surface $\rightarrow$ có thể đi qua noisy / high-curvature regions.
- **ManifoldAligned Guided Integrated Gradients (MA-GIG):** xây dựng attribution paths trong latent space của một variational autoencoder được pre-trained $\rightarrow$ Bằng cách decoding các intermediate latent states, MA-GIG định hướng path về phía generative manifold đã được học và giảm việc đi qua các vùng không hợp lý trong input space.
- $\rightarrow$ MA-GIG tạo ra faithful explanations bằng cách tổng hợp các gradients trên các path features nằm gần input $\rightarrow$ giảm off-manifold noise và vượt trội so với các path-based attribution methods trước đây trên nhiều datasets và classifiers.

2 Contributions

- Path đi vào những vùng dữ liệu “không tự nhiên” $\rightarrow$ gradient ở đó có thể nhiều $\rightarrow$ attribution bị sai.
- Xây dựng Manifold-Aligned Guided Integrated Gradients. Tận dụng latent space của generative model + decoder để định hướng integration path về learned data manifold. Thay vì tính gradient trên các ảnh trung gian được tạo ra bởi interpolation và có thể không giống dữ liệu thực.
IG: Input thật $\rightarrow$ baseline (ví dụ ảnh đen) $\rightarrow$ tạo các intermediate inputs bằng interpolation $\rightarrow$ tính gradient trên từng intermediate input $\rightarrow$ tổng hợp attribution.
MA-GIG: Input thật $\rightarrow$ đi qua latent space $\rightarrow$ decoder tạo intermediate inputs $\rightarrow$ tính gradient trên các intermediate inputs này $\rightarrow$ tổng hợp attribution.
- Qua qualitative analyses + quantitative evaluations, cho thấy MA-GIG tạo ra attribution maps: perceptually aligned hơn, reliable hơn

3 Motivation

Note: data manifold - Giả sử input space có rất nhiều chiều. Với ảnh $224 \times 224 \times 3$, có hơn 150,000 pixel values, nên về mặt toán học có vô số ảnh có thể tạo ra. Nhưng ảnh tự nhiên chỉ chiếm một phần cực nhỏ trong không gian đó $\rightarrow$ data manifold .

3.1 Problem Formulation

- Có classifier $f(x)$ dự đoán xác suất của một class.
- **Feature attribution** xác định mức độ đóng góp của từng feature x_i vào prediction.
- So sánh input thật x với **baseline** x' , ví dụ **zero vector** / **ảnh đen**.
- Kết quả là attribution map A , cho biết feature nào ảnh hưởng nhiều và theo hướng nào.

3.2 Path Methods

Ý tưởng chung:

$$x' \rightarrow \text{path} \rightarrow x$$

- Thay vì tính attribution chỉ tại một điểm, path-based methods **tích lũy gradient dọc theo một path** từ baseline đến input.
- Nhờ đó thỏa mãn **Completeness**:

$$\sum_i A_i = f(x) - f(x')$$

IG:

- Chọn **straight-line path** từ baseline $\rightarrow$ input.
- Tạo các intermediate inputs bằng linear interpolation (Lấy baseline (ví dụ ảnh đen) và input thật, rồi tạo ra nhiều ảnh nằm ở giữa hai ảnh đó bằng cách thay đổi dần từ baseline sang input.)
- Tính gradient tại các điểm này rồi cộng/tích lũy lại.

GIG:

- chứng minh straight-line path có thể đi qua vùng có **gradient noise**.
- Tạo **adaptive path**, ưu tiên cập nhật feature có gradient magnitude nhỏ.
- Mục tiêu: đi qua những vùng prediction surface phẳng hơn $\rightarrow$ giảm **spurious gradients**.

3.3 Manifold Hypothesis

Đây là motivation cho MA-GIG:

- Dữ liệu thật được giả định nằm trên một **data manifold** $\mathcal{M}$ trong input space.
- Các hướng thay đổi tự nhiên của dữ liệu nằm trong **tangent space** $T_x\mathcal{M}$.
Note: Nếu đang đứng tại một ảnh x trên manifold, tangent space là những hướng mà có thể thay đổi ảnh một cách tự nhiên mà vẫn gần manifold.
- Nếu path cập nhật theo hướng nằm ngoài tangent space:

$$\Delta x \notin T_x\mathcal{M}$$

$\rightarrow$ điểm có thể đi **off-manifold** $\rightarrow$ classifier behavior và gradient ở đó có thể không đáng tin.

$\rightarrow$ đề xuất **manifold-aligned attribution path**.

4 Manifold-Aligned Guided IG

4.1 Tại sao GIG dù cố giảm gradient noise nhưng vẫn có thể đi lệch khỏi data manifold?

GIG chọn một số pixel/features có gradient magnitude nhỏ rồi thay đổi chúng từng bước. Do đó, update của GIG là **sparse và axis-aligned**, tức là di chuyển theo các coordinate axes của input space, chứ không nhất thiết theo hướng tự nhiên của data manifold.

Tuy nhiên, data manifold có thể cong và phức tạp. Để một update vẫn **manifold-aligned** ở mức xấp xỉ bậc nhất, vector thay đổi cần nằm trong tangent space:

$$\Delta x^{(k)} \in T_{x^{(k)}} \mathcal{M}.$$

GIG không đảm bảo điều kiện này.

Paper tách update thành hai thành phần:

$$\Delta x^{(k)} = \Delta x_{\parallel}^{(k)} + \Delta x_{\perp}^{(k)}$$

Trong đó:

$$\Delta x_{\parallel}^{(k)} = \text{proj}_{T_{x^{(k)}} \mathcal{M}} \left(\Delta x^{(k)} \right)$$

là thành phần theo tangent space, còn

$$\Delta x_{\perp}^{(k)}$$

là thành phần vuông góc với tangent space và được xem là **manifold deviation error**.

Một bước lệch khỏi manifold có thể nhỏ. Tuy nhiên, GIG thực hiện nhiều bước liên tiếp:

$$x^{(0)} \rightarrow x^{(1)} \rightarrow x^{(2)} \rightarrow \dots \rightarrow x^{(K)}.$$

Mỗi bước có thể tạo ra một thành phần lệch khỏi manifold:

$$\Delta x_{\perp}^{(k)}.$$

Các sai lệch này có thể tích lũy theo toàn bộ path:

$$d(x^{(K)}, \mathcal{M}) \leq \sum_{k=0}^{K-1} \left\| \Delta x_{\perp}^{(k)} \right\| + O(\kappa).$$

Do đó, càng thực hiện nhiều bước, path càng có nguy cơ **drift khỏi data manifold**, khiến gradient được tính tại những vùng có ít hoặc không có dữ liệu hỗ trợ.

GIG chọn feature tốt về gradient $\nrightarrow$ path tốt về manifold

4.2 Manifold Alignment via Latent Guidance

GIG làm guidance trong input space:

$$x' \rightarrow x$$

nên update theo các pixel/features có thể bị off-manifold.

MA-GIG đổi sang latent space:

$$z' \rightarrow z$$

rồi dùng decoder D để chuyển latent representation thành input:

$$z \xrightarrow{D} x$$

Update trong latent space tuy vẫn là axis-aligned, nhưng khi đi qua decoder, nó trở thành một hướng có cấu trúc trong input space và được định hướng theo tangent space của manifold.

Bước 1 — Encode Input thật và baseline được đưa vào encoder:

$$z = E(x), \quad z' = E(x')$$

Sau đó bắt đầu tại:

$$z^{(0)} = z'$$

Tức là bắt đầu từ baseline trong latent space.

Bước 3 — Tính gradient trong latent space Thay vì tính:

$$\nabla_x f(x)$$

rồi chọn pixel như GIG, MA-GIG tính:

$$\nabla_z f(D(z)) = J_D(z)^T \nabla_x f(D(z))$$

MA-GIG xem mỗi latent dimension ảnh hưởng thế nào đến prediction.

Sau đó chọn những latent dimensions có gradient magnitude nhỏ.

Note: Jacobian Jacobian là một ma trận chứa các đạo hàm riêng, dùng để mô tả một hàm nhiều biến thay đổi như thế nào khi đầu vào thay đổi.

Giả sử decoder D biến latent vector:

$$z = \begin{bmatrix} z_1 \\ z_2 \end{bmatrix}$$

thành input:

$$x = D(z) = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}.$$

Jacobian của decoder là:

$$J_D(z) = \begin{bmatrix} \frac{\partial x_1}{\partial z_1} & \frac{\partial x_1}{\partial z_2} \\ \frac{\partial x_2}{\partial z_1} & \frac{\partial x_2}{\partial z_2} \\ \frac{\partial x_3}{\partial z_1} & \frac{\partial x_3}{\partial z_2} \end{bmatrix}$$

Nó có shape:

$$n \times d$$

$z_1 \rightarrow$ thay đổi hình dạng

$z_2 \rightarrow$ thay đổi màu sắc

Khi tăng z_1 một chút, rất nhiều pixel trong ảnh có thể thay đổi.

Jacobian cho biết:

Nếu thay đổi từng latent dimension một chút, thì từng pixel/input feature sẽ thay đổi bao nhiêu.

Ví dụ cột thứ nhất:

$$\frac{\partial D}{\partial z_1} = \begin{bmatrix} \frac{\partial x_1}{\partial z_1} \\ \frac{\partial x_2}{\partial z_1} \\ \vdots \\ \frac{\partial x_n}{\partial z_1} \end{bmatrix}$$

là hướng thay đổi của toàn bộ input khi z_1 thay đổi.

GIG:

thay đổi pixel $\rightarrow$ có thể lệch manifold

MA-GIG:

thay đổi latent $\rightarrow J_D \rightarrow$ thay đổi nhiều pixel có cấu trúc $\rightarrow$ gần manifold hơn

Jacobian = cầu nối giữa latent space và input space

$$\Delta z \xrightarrow{J_D} \Delta x$$

Bước 4 — Update latent dimensions Chỉ update những latent dimensions được chọn:

$$\Delta z^{(k)}$$

Dù Δz là axis-aligned trong latent space, sau khi đi qua decoder, nó không còn axis-aligned trong input space.

4.3 So sánh GIG và MA-GIG

GIG

gradient $\rightarrow$ chọn pixel $\rightarrow \Delta x = \delta_i e_i \rightarrow$ axis-aligned trong input space $\rightarrow$ có thể lệch khỏi manifold

MA-GIG

gradient $\rightarrow$ chọn latent dimensions $\rightarrow \Delta z = \delta_j u_j \rightarrow$ Decoder $\rightarrow \Delta x \approx J_D(z) \Delta z \rightarrow$ tangent direction $\rightarrow$ gần / n

4.4 Perfect Autoencoder Assumption

Paper giả định decoder D rất tốt:

$$D(E(x)) = x$$

với mọi x trên manifold.

Và:

$$D(\mathcal{Z}) = \mathcal{M}$$

nghĩa là latent space khi đi qua decoder tạo ra data manifold.

Do đó:

$$z(t) \in \mathcal{Z} \Rightarrow D(z(t)) \in \mathcal{M}$$

Theo giả định lý tưởng này, continuous latent path khi decode sẽ nằm trên manifold.

VAE thực tế không vậy. Nên MA-GIG không thể đảm bảo tuyệt đối:

“Mọi intermediate input chắc chắn nằm trên true data manifold.”

Thay vào đó, decoder cung cấp một generative prior, giúp intermediate samples plausible hơn và gần learned manifold hơn.

5 Experimental Setup

test trên 3 datasets:

- **ImageNet**: 1,000 classes.
- **Oxford-IIIT Pet**: 37 classes, nhiều class khá giống nhau $\rightarrow$ test fine-grained classification.
- **Oxford 102 Flower**: 102 classes, khác nhau về scale, pose, background.

Classifier gồm:

- VGG16
- ResNet18
- InceptionV1

Các model này bắt đầu từ ImageNet pretrained weights. Với Oxford Pet/Flower thì fine-tune lại trên training set tương ứng.

Baselines Họ so sánh MA-GIG với nhiều attribution methods:

Gradient-based:

$$G \times I$$

Path-based:

$$IG, IG2, GIG, AGI$$

Trong đó:

- **IG**: straight-line path.
- **IG2**: tìm path có gradient dốc hơn.
- **GIG**: adaptive path, chọn feature có gradient nhỏ.
- **AGI**: đi theo steepest-ascent path từ adversarial examples.

Latent/manifold-based:

$$EIG, MIG$$

- **EIG**: interpolation trong VAE latent space.
- **MIG**: geodesic path trong VAE latent space.

Và MA-GIG cũng thuộc nhóm latent-space nhưng kết hợp latent guidance của GIG + decoder geometry.

5.1 Qualitative Comparison

→ Các baseline thường tạo attribution phân tán và có nhiều background noise ở Figure 2.

MA-GIG cho attribution tập trung hơn vào class-relevant regions.

5.2 Quantitative Evaluation

DiffID

Để đánh giá faithfulness của attribution.

Ngoài ra có:

Insertion:

Ban đầu che hết → lần lượt reveal pixel quan trọng.

Nếu attribution tốt:

confidence ↑ nhanh

Deletion:

Ban đầu ảnh đầy đủ → lần lượt remove pixel quan trọng.

Nếu attribution tốt:

confidence ↓ nhanh

Xem Table 1: MA-GIG vượt. Và họ còn test với nhiều VAE backbones ở Appendix.

MAR — Mask-Aware Reconstruction

SD1 — Stable Diffusion 1.x

SD2 — Stable Diffusion 2.x

KD — Knowledge Distillation

MAR và SD2 thường cho kết quả robust nhất vì MAR được pre-trained specifically on ImageNet.

Trong khi SD2 được train trên:

LAION5B

một dataset cực lớn.

Nhưng dù SD2 có generative model lớn hơn, MAR vẫn thường:

$MAR \geq SD2$

trong một số benchmark.

→ Generative model lớn hơn không đồng nghĩa attribution tốt hơn.

Kết luận là performance không chỉ tốt với một generative backbone duy nhất.

5.3 Manifold Alignment Analysis

kiểm tra: Intermediate samples của MA-GIG có thực sự nhất quán về mặt tri giác (perceptually coherent) hơn GIG không?

LPIPS (Learned Perceptual Image Patch Similarity) để đo khoảng cách giữa intermediate sample và input cuối cùng:

$$LPIPS(\gamma(\alpha), x)$$

$\gamma(\alpha)$: intermediate sample tại vị trí α .

x : input thật.

$\alpha = 0$: baseline

$\alpha = 1$: input

Ý tưởng chính:

x_1, x_2 được đưa qua một pre-trained neural network để lấy feature:

$$\phi_l(x_1), \quad \phi_l(x_2)$$

Ở mỗi layer l , lấy feature map rồi chuẩn hóa theo channel:

$$\hat{\phi}_l(x) = \frac{\phi_l(x)}{\|\phi_l(x)\|_2}$$

Sau đó tính difference:

$$d_l = \left\| \hat{\phi}_l(x_1) - \hat{\phi}_l(x_2) \right\|^2$$

LPIPS kết hợp các difference ở nhiều layers:

$$LPIPS(x_1, x_2) = \sum_l \frac{1}{H_l W_l} \sum_{h,w} \left\| w_l \odot \left(\hat{\phi}_l(x_1)_{hw} - \hat{\phi}_l(x_2)_{hw} \right) \right\|_2^2$$

Trong đó w_l là learned weights.

Không cần bạn tự train network này; LPIPS sử dụng pretrained feature network + learned linear calibration.

LPIPS không phải manifold distance trực tiếp:

Figure 3

→ GIG có intermediate images thay đổi perceptually mạnh/không ổn định hơn.

→ MA-GIG có intermediate samples coherent hơn về mặt perceptual.

5.4 Effect of Selection Fraction

kiểm tra parameter:

$$q$$

q là fraction latent dimensions được chọn để update.

test:

$$q \in \{0.05, 0.1, 0.2\}$$

Ví dụ:

$$q = 0.05$$

→ mỗi step chỉ chọn khoảng 5% latent dimensions có gradient magnitude nhỏ nhất.

Họ quan sát DiffID.

Kết quả:

DiffID khá ổn định khi thay đổi q .

Và quan trọng:

$$\text{MA-GIG} > \text{GIG}$$

trên tất cả datasets và classifiers được test.

Đặc biệt:

$$q = 0.05$$

thường cho performance tốt nhất.

Ý nghĩa MA-GIG không quá nhạy với hyperparameter q .

Tức là không cần phải tìm một q cực kỳ chính xác mới hoạt động.

5.5 Reconstruction MSE vs Attribution Quality

Có phải VAE nào reconstruct ảnh tốt hơn thì MA-GIG cũng cho attribution tốt hơn không?

Reconstruction:

$$\text{MSE}(x, D(E(x)))$$

và:

Attribution quality:

$$\text{DiffID}$$

Kết quả:

Không nhất thiết

Table 2

KD có:

lowest MSE

tức là reconstruct ảnh tốt nhất.

Nhưng lại có:

lowest DiffID

→ attribution không tốt nhất.

Trong khi:

MAR + SD2

có attribution tốt hơn.

Nếu chỉ nhìn reconstruction:

VAE tốt $\iff$ reconstruction MSE thấp

nhưng đối với MA-GIG:

reconstruction quality $\neq$ attribution quality

Bởi vì MA-GIG không chỉ cần decoder reconstruct ảnh tốt.

Nó cần latent space có:

task-relevant latent structure

và:

domain alignment

5.6 Effect of Slerp

Linear interpolation:

$$z(t) = (1 - t)z' + tz$$

Slerp:

Spherical Linear Interpolation

đi theo đường cong trên hypersphere thay vì đường thẳng thông thường.

Slerp có làm latent path tốt hơn không?

Kết quả:

Mixed results

Table 5

Có setting Slerp tốt hơn, nhưng có setting lại tệ hơn.
Ví dụ OxfordPet + ResNet18:

Linear = 0.4637

Slerp = 0.4565

→ Slerp làm giảm performance ở setting này.
Do đó họ chọn:

Linear interpolation = default

vì:

- đơn giản hơn
- performance tương đương
- Slerp không đem lại improvement nhất quán.